BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC SƯ PHẠM KĨ THUẬT TP.HCM

BÁO CÁO

MÔN HỌC : XỬ LÝ ẢNH

ĐỀ TÀI: HỆ THỐNG CHỐNG NGỦ GẬT TRÊN Ô TÔ KẾT HỢP VỚI NHẬN DIỆN BÀN TAY

GVHD : TS. NGUYỄN VĂN THÁI

Nhóm thực hiện : Trần Minh Hoàng 21151236

Nguyễn Minh Chiến 21151198

Báo Ngọc Đăng Khoa 21151408

TP.Thủ Đức, tháng 06 năm 2023

LỜI CAM ĐOAN

Đề tài này là do nhóm dựa trên tài liệu tham khảo trên internet và kết hợp phát triển với các tính năng tiện ích không sao chép hoàn toàn bất cứ tài liệu nào.

BẢNG PHÂN CÔNG NHIỆM VỤ

STT	HỌ VÀ TÊN	MSSV	NHIỆM VỤ	TIẾN ĐỘ
1	Trần Minh Hoàng	21151236	Lập trình giao diện, nhận diện khuôn miệng, các đồ thị theo dõi và tổng hợp Word.	100%
2	Nguyễn Minh Chiến	21151198	Lập trình nhận diện và đếm số ngón tay, chỉnh sửa tổng hợp code.	100%
3	Báo Ngọc Đăng Khoa	21151408	Lập trình đếm số lần nhắm mắt, nghiên cứu tài liệu các thư viện.	100%

Ghi chú:

NHẬN XÉT CỦA GIẢNG VIÊN

.....
.....
.....
.....
.....
.....
.....
.....
.....
.....

Điểm:.....

Chữ kí GVHD

TS. Nguyễn Văn Thái

MỤC LỤC

CHƯƠNG 1 : GIỚI THIỆU TỔNG QUAN ĐỀ TÀI XỬ LÝ ẢNH 5

1.1 Giới thiệu : 5

1.2 Đặt vấn đề :.....6

1.3 Mục tiêu : 6

1.4 Phương pháp nghiên cứu : 6

1.5 Giới hạn đề tài : 7

CHƯƠNG 2 : CƠ SỞ LÝ THUYẾT 8

2.1 Lý thuyết xử lý ảnh : 8

2.2 Các thư viện thường gặp :.....8

2.2.1 PILOW Xử lý hình ảnh :.....8

2.2.2 Matplotlib – Vẽ đồ thị 2D:.....9

2.2.3 Numpy – Xử lý mảng đa chiều, ma trận:.....9

2.2.4 OpenCV – Xử lý hình ảnh:.....10

2.2.5 Requests – Thư viện HTTP:.....10

2.2.6 Keras – Deep Learning: 11

CHƯƠNG 3 : NỘI DUNG THỰC HIỆN 12

3.1 Yêu cầu đề tài : 12

3.2 Lưu đồ giải thuật :.....12

CHƯƠNG 4 : CODE VÀ GIẢI THÍCH HOẠT ĐỘNG CHƯƠNG TRÌNH 14

4.1 Giao diện ban đầu :.....14

4.2 Nhận diện khuôn mặt và đếm số lần nháy mắt:.....15

4.2.1 Đếm số lần nhắm mắt : 15

Bước đầu ta sẽ khai báo các thư viện :.....15

4.2.2 Phát hiện trạng thái mở miệng và mức độ mở miệng (ngáp) :.....19

4.2.3 Các thư viện :.....20

4.3 Phương pháp dùng để thực hiện chương trình :.....21

❖ Phương pháp nhận dạng dựa trên các đặc trưng khuôn mặt 21

4.4 Nhận diện đếm số ngón tay :.....25

CHƯƠNG 5 : KẾT QUẢ VÀ KẾT LUẬN.....32

5.1 Vấn đề gặp phải và hướng giải quyết :.....32

5.2 Kết quả đạt được :.....33

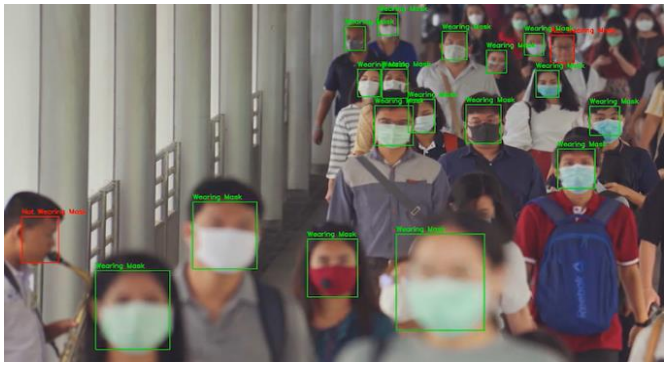
5.3 Hạn chế của đề tài :.....34

5.4 Hướng phát triển :.....34

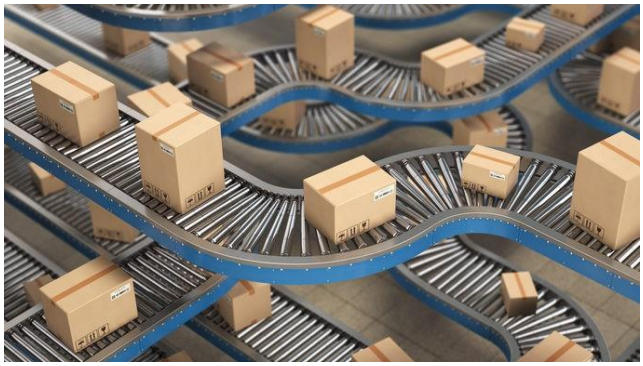
CHƯƠNG 1 : GIỚI THIỆU TỔNG QUAN ĐỀ TÀI XỬ LÝ ẢNH

1.1 Giới thiệu :

Từ xưa đến nay con người chúng ta luôn luôn quan sát và thu thập thông tin phần lớn thông qua hình ảnh. Vì thế công nghệ xử lí ảnh ra đời có vai trò vô cùng quan trọng trong nền văn minh nhân loại. Nó xử lí được chính xác và có thể phát hiện được những cấu trúc nhỏ mà con người không thể quan sát được. Đã có nhiều phát minh có giá trị to lớn trong hầu hết lĩnh vực: y tế, quân sự, khoa học … Điển hình như một số ứng dụng nhận dạng khuôn mặt, nhận dạng các vật thể, xử lí nhiễu, nhận dạng hành động .

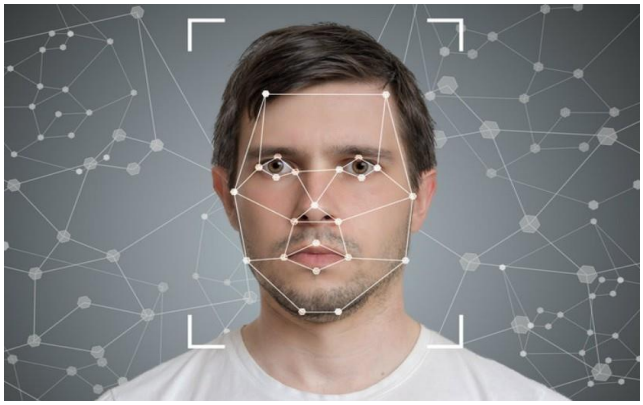


Hình 1.1 Nhận diện người



Hình 1.2 Phân loại sản phẩm

Hình 1.3 Nhận diện biển số xe



Hình 1.4 Nhận diện khuôn mặt

1.2 Đặt vấn đề :

Ngày nay, xã hội ngày càng phát triển. Công nghiệp hóa, hiện đại hóa ngày càng được nâng cao để phát triển đất nước và cải thiện cuộc sống người dân. Vì vậy việc ứng dụng khoa học kỹ thuật ngày càng trở nên rộng rãi, phổ biến và mang lại hiệu quả cao trong hầu hết các lịnh vực kinh tế, kỹ thuật cũng như trong đời sống xã hội.Thị giác máy là một lĩnh vực đã và đang phát triển. Thị giác máy bao gồm lý thuyết và các kỹ thuật liên quan đến mục đích nhằm tạo ra một hệ thống nhân tạo có thể tiếp nhận thông tin từ các hình ảnh thu được hoặc các tập dữ liệu đa chiều. Việc kết hợp giữa thị giác máy với các kỹ thuật khác như truyền thông, điện tử, điều khiển tự động, cơ khí….cho chúng ta rất nhiều ứng dụng trong hàng ngày cũng như trong khoa học, an ninh, quân sự…..Trong đó, hiện nay đất nước ta đang khuyến khích người dân chuyển đổi từ chứng minh nhân dân sang căn cước công dân có gắn chip để thuận lợi hơn cho nhà nước trong quá trình quản lý dân cư. Cộng thêm quá trình học tập môn xử lý ảnh chúng em được tiếp cận ban đầu và nghiên cứu về xử lý ảnh số sử dụng mã nguồn OpenCV, ngôn ngữ C#, python. Từ đó chúng em quyết định nghiên cứu về đề tài "Hệ thống chống ngủ gật trên xe ô tô kết hợp với nhận diện bàn tay" với sự hướng dẫn của thầy T.s Nguyễn Văn Thái nhóm chúng em đã nghiên cứu thành công và mong muốn tiếp tục phát triển ứng dụng vào các lĩnh vực khác.....

1.3 Mục tiêu :

Đề tài: "Hệ thống chống ngủ gật trên xe ô tô kết hợp với nhận diện bàn tay" với mục tiêu là giúp tài xế tránh được tình trạng ngủ gật làm nguy hiểm trong quá trình tham gia giao thông. Hệ thống cảnh báo khi có dấu hiệu buồn ngủ và tài xế có thể lựa chọn các chế độ bằng cử chỉ bàn tay.

1.4 Phương pháp nghiên cứu :

Vận dụng các kiến thức đã học và tham khảo các mô hình trên internet để tìm ra phương pháp tối ưu cho đề tài.

1.5 Giới hạn đề tài :

- Chỉ hiển thị được số lần đếm, chưa hiển thị được tần suất.
- Chưa có thông tin từ tần suất chớp mắt (chưa áp dụng được tâm lí học chớp mắt vào để tài).
- Không đếm được ngoài khoảng cách cho phép(khoảng cách từ đối tượng đến camera).
- Bàn tay nhận tối đa 5 ngón.
- Khi phát âm thanh, giao diện phải chờ hết âm thanh mới thực hiện tiếp.
- Không phát hiện nhiều đối tượng.

CHƯƠNG 2 : CƠ SỞ LÝ THUYẾT

2.1 Lý thuyết xử lý ảnh :

Xử lý ảnh (XLA) là đối tượng nghiên cứu của lĩnh vực thị giác máy, là quá trình biến đổi từ một ảnh ban đầu sang một ảnh mới với các đặc tính và tuân theo ý muốn của người sử dụng. Xử lý ảnh có thể gồm quá trình phân tích, phân lớp các đối tượng, làm tăng chất lượng, phân đoạn và tách cạnh, gán nhãn cho vùng hay quá trình biên dịch các thông tin hình ảnh của ảnh. Cũng như xử lý dữ liệu bằng đồ hoạ, xử lý ảnh số là một lĩnh vực của tin học ứng dụng. Xử lý dữ liệu bằng đồ họa đề cập đến những ảnh nhân tạo, các ảnh này được xem xét như là một cấu trúc dữ liệu và được tạo bởi các chương trình. Xử lý ảnh số bao gồm các phương pháp và kỹ thuật biến đổi, để truyền tải hoặc mã hoá các ảnh tự nhiên. Mục đích của xử lý ảnh gồm:

- Biến đổi ảnh làm tăng chất lượng ảnh.
- Tự động nhận dạng ảnh, đoán nhận ảnh, đánh giá các nội dung của ảnh.

Nhận biết và đánh giá các nội dung của ảnh là sự phân tích một hình ảnh thành những phần có ý nghĩa để phân biệt đối tượng này với đối tượng khác, dựa vào đó ta có thể mô tả cấu trúc của hình ảnh ban đầu. Có thể liệt kê một số phương pháp nhận dạng cơ bản như nhận dạng ảnh của các đối tượng trên ảnh, tách cạnh, phân đoạn hình ảnh,… Kỹ thuật này được dùng nhiều trong y học (xử lý tế bào, nhiễm sắc thể), nhận dạng chữ trong văn bản.Module tiết diện:

2.2 Các thư viện thường gặp :

2.2.1 PILOW Xử lý hình ảnh :

Khi sử dụng Pilow, người dùng có thể mở và lưu hình ảnh, mà còn có thể xử lý đặc điểm của hình ảnh, chẳng hạn như màu sắc, độ mờ, độ sáng, tối, ... Pilow hỗ trợ xử lý nhiều tệp hình ảnh khác nhau như PDF, WebP, PCX, PNG, JPEG, GIF, PSD, WebP, PCX, GIF, IM, EPS, ICO, BMP, và còn nhiều hơn thế.

Với Pilou, bạn có thể dễ dàng tạo ra những ảnh thu nhỏ (thumbnails) cho hình ảnh, những ảnh thu nhỏ này mang hầu hết đặc điểm của hình ảnh và gần như không khác gì ảnh gốc ngoại trừ chúng được thu nhỏ lại. Pillow hỗ trợ một bộ sưu tập các bộ lọc hình ảnh như – FIND_EDGES, DETAIL, SMOOTH, BLUR, CONTOUR, SHARPEN, SMOOTH_MORE, etc.

2.2.2 Matplotlib – Vẽ đồ thị 2D:

Matplotlib có thể tạo ra những đồ thị chất lượng và xuất ra một cách dễ dàng và thuận tiện, hoàn toàn đáp ứng nhu cầu của mọi ngành học. Các đồ thị được tạo ra bằng Matplotlib có sẵn bản sao cứng trên các nền tảng tương tác khác nhau.

Bạn có thể dùng Matplotlib với nhiều bộ công cụ như Python Scripts, IPython Shells, Jupyter Notebook, và nhiều công cụ khác. Một số thư viện của bên thứ ba có thể được tích hợp với các ứng dụng Matplotlib. Chẳng hạn như seaborn, ggplot, và các bộ công cụ chiếu xạ, mapping khác như basemap.

Ngoài ra, bạn còn có thể theo dõi bất kỳ lỗi nào phát sinh trong quá trình coding, các bản vá mới, đồng thời còn có thể đóng góp các tính năng mới tại GitHub. Đó là một trang chính thức để nêu ra các vấn đề liên quan đến Matplotlib và cùng giải quyết chúng.

2.2.3 Numpy – Xử lý mảng đa chiều, ma trận:

Numpy là một mô-đun mở rộng mã nguồn mở cho Python, cung cấp các chức năng biên dịch nhanh cho các thao tác toán học và số, thậm chí là với những ma trận và mảng có lượng dữ liệu khổng lồ. Bên cạnh đó các mô-đun cung cấp một thư viện lớn các chức năng toán học cấp cao để hoạt động trên các ma trận và mảng một cách dễ dàng và thuận tiện.

Numpy cung cấp những masked arrays đồng thời với mảng gốc. Nó cũng đi kèm với các chức năng như thao tác với hình dạng logic, biến đổi Fourier rời rạc, đại số

tuyến tính tổng quát, và nhiều hơn nữa. Gói mô-đun này cung cấp các công cụ hữu ích để tích hợp với các ngôn ngữ lập trình khác. Chẳng hạn như C, C++, và ngôn ngữ lập trình Fortran. Numpy cung cấp các chức năng tương đương với MATLAB. Cả hai đều cho phép người dùng thao tác nhanh hơn.

2.2.4 OpenCV – Xử lý hình ảnh:

OpenCV là một gói mô-đun hình ảnh lý tưởng cho phép bạn đọc và ghi, thay đổi dữ liệu nhiều hình ảnh cùng một lúc. Tạo ra thị giác máy tính cho phép bạn xây dựng lại, gián đoạn và thông hiểu môi trường 3D từ môi trường 2D tương ứng của nó. OpenCV được xử dùng nhiều trong nhận diện vật thể và hình ảnh được thiết lập trước, chẳng hạn như khuôn mặt, động vật, cây cối, các vật thể di chuyển, etc. Bạn cũng có thể lưu và chụp bất kỳ khoảnh khắc nào của video và cũng có thể phân tích các thuộc tính khác nhau của nó như chuyển động, nền, etc. OpenCV tương thích với nhiều hệ điều hành như Windows, OS-X, Open BSD và nhiều hệ điều hành khác.

2.2.5 Requests – Thư viện HTTP:

Sử dụng Từ điển Python cơ bản trong Requests, bạn có thể thêm các tham số, tiêu đề, multi-part file và dữ liệu biểu mẫu. Đây là một thư viện dễ dàng với rất nhiều tính năng cho phép bạn xử lý các tiêu đề tùy chỉnh, xác minh chứng chỉ SSL và quét các tham số về URL. Với Requests, bạn có thể dễ dàng tải lên nhiều tệp cùng một lúc. Nó cho phép bạn làm việc trong một môi trường nhanh hơn và hiệu quả hơn.

Requests có tính năng giải nén tự động cho phép bạn khôi phục và phục hồi dữ liệu đã nén thành dạng xác thực của nó ngay lập tức. Tận hưởng những lợi ích của hỗ trợ proxy HTTP với Requests. Và cho phép người dùng của bạn một tuyến nhanh hơn và đơn giản hơn đến các tệp và trang của bạn. Requests cũng có các cookie giá trị, cơ quan phản hồi Unicode, xác thực Basic/Digest, an toàn luồng, tổng hợp kết nối và nhiều hơn thế nữa..

2.2.6 Keras – Deep Learning:

Keras là một thư viện Python mạnh mẽ. Nó có khả năng chạy trên Bộ công cụ nhận thức của Microsoft (Microsoft Cognitive Toolkit), PaidML, TensorFlow và các nền tảng khác. Thư viện Python này có nhiều cách triển khai từ các khối hình thành mạng nơ-ron - chức năng, lớp, trình tối ưu hóa, các đối tượng và những thứ khác. Keras cũng có nhiều công cụ hữu ích cho phép bạn làm việc với các hình ảnh và văn bản khác nhau một cách dễ dàng. Nó không chỉ hỗ trợ mạng nơ-ron mà còn cung cấp một môi trường hỗ trợ đầy đủ cho các mạng nơ-ron tích hợp và tái hiện.

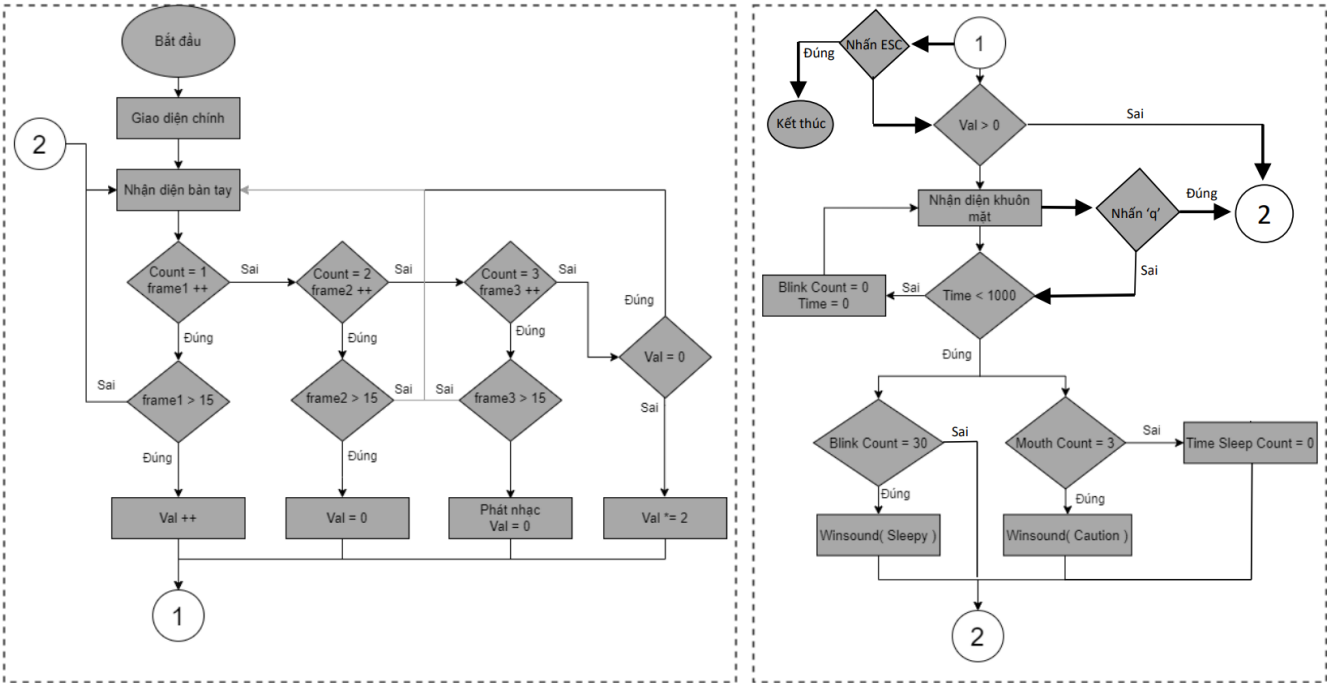
Sử dụng Keras, bạn có thể xây dựng các mô hình sâu cho điện thoại thông minh - cả Android và iOS hoặc cả Java Virtual Machine.

CHƯƠNG 3 : NỘI DUNG THỰC HIỆN

3.1 Yêu cầu đề tài :

Nhận diện khuôn mặt và đếm số lần chợp mắt thông qua độ dài hai mí mắt kết hợp với nhận diện bàn tay để chọn chế độ của chương trình.

3.2 Lưu đồ giải thuật :



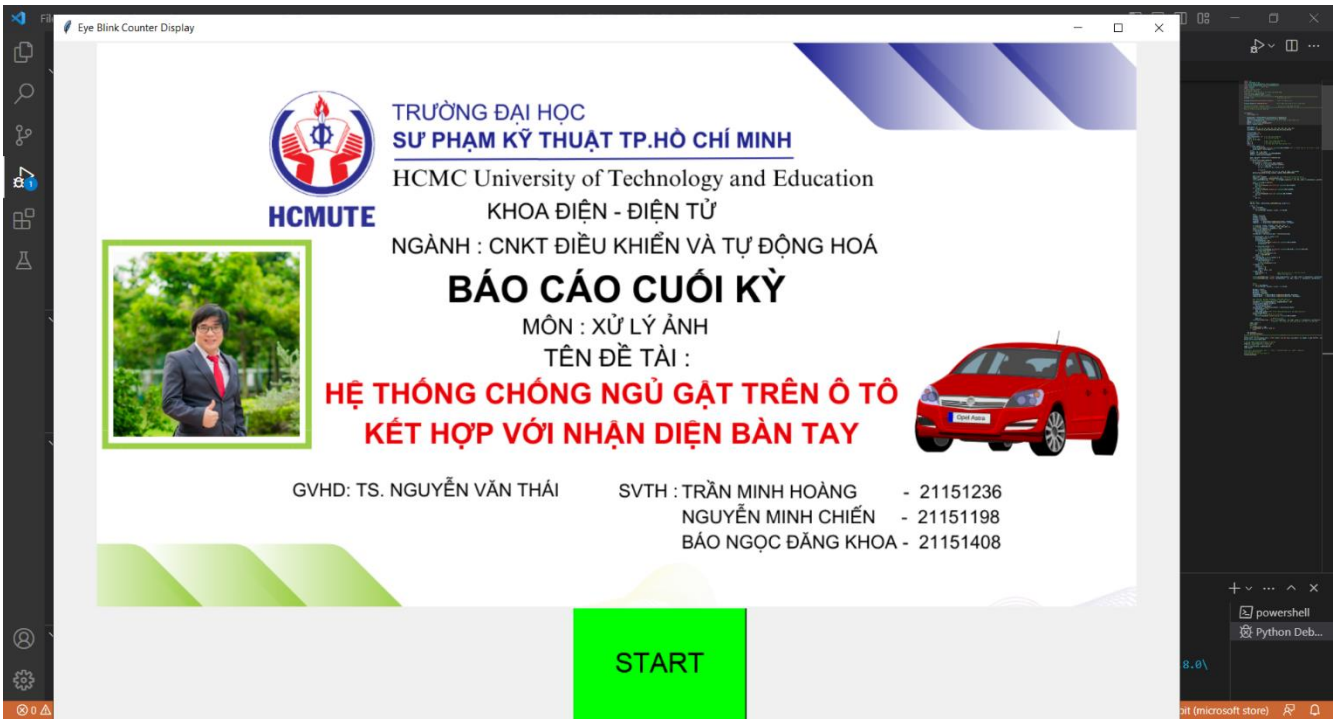
Hình 3.2.1 Lưu đồ thuật toán của chương trình

Bắt đầu chương trình sẽ hiển thị giao diện chính kèm theo button "START". Khi ra nhấn button thì chương trình nhận diện bàn tay được thực hiện. Nếu bàn tay hợp lệ (số ngón tay không quá 5), thì chương trình sẽ đếm số ngón tay với các trường hợp 1,2,3 và các trường hợp còn lại. Trường hợp 1 sẽ kích hoạt hàm nhận diện khuôn mặt, trường hợp 2 sẽ kết thúc hàm nhận diện khuôn mặt (nếu có), trường hợp 3 sẽ phát nhạc sôi động giúp tài xế tỉnh táo hơn. Trường hợp còn lại sẽ có chức năng giữ nguyên trạng thái biến đếm (giúp kích hoạt 1 lần mà không cần phải giơ tay tiếp).

Về hàm nhận diện mặt, đầu tiên biến đếm thời gian sẽ bắt đầu. Trong vòng t= 1000 (vòng lặp) nếu tài xế chớp mắt quá 30 lần/t thì hệ thống sẽ phát âm thanh nhắc nhở. Nếu tài xế chớp mắt ít hơn 30 lần /t thì biến BlinkCount sẽ reset về 0 giúp tăng khả năng xác định ngủ gật. Kế tiếp đó là biến timesleep, nếu tài xế nhắm mắt trong vòng 3 giây thì hệ thống sẽ phát âm thanh cảnh báo nguy hiểm. Cuối cùng đó là nhận diện tài xế ngáp, nếu tài xế ngáp quá 3 lần /t thì hệ thống sẽ phát âm thanh mời chọn lại chế độ, khuyên tài xế dừng lại nghỉ ngơi. Do khi ngáp, mắt người thường khép lại dẫn đến biến timesleep tăng lên gây cảnh báo làm khó chịu nên nhóm em đã gán timesleep = 0 khi tài xế ngáp giúp tránh tình trạng gây khó chịu. Cuối cùng chương trình sẽ quay về hàm tay tiếp tục vòng lặp tiếp theo.

CHƯƠNG 4 : CODE VÀ GIẢI THÍCH HOẠT ĐỘNG CHƯƠNG TRÌNH

4.1 Giao diện ban đầu :



Hình 4.1.1 Giao diện của chương trình

```
#=========================================GIAO DIỆN==================================================
display = Tk()                                  #mở giao diện chính

display.title('Eye Blink Counter Display')      #Tên của chương trình

display.geometry('1300x800+50+10')             #Kích thước giao diện & vị trí xuất hiện

#display.attributes('-topmost',True)             #Làm cửa sổ luôn hiện trên top
```

Hình 4.1.2 Cấu hình giao diện

Sử dụng thư viện tkinter, mở giao diện chính và cài đặt các thông số cho giao diện như : Tên chương trình, kích thước giao diện, vị trí xuất hiện,…

```
#==================================================Button==================================================
#Tạo nút nhấn bắt đầu
button_start = Button(display,text = 'START',font=('Time New Roman',24),width = 10, height = 3,bg='#00FF00', command = Start)
button_start.place(x=600,y=650)

# Tạo đối tượng ImageTk.PhotoImage từ ảnh PIL
image_tk = PhotoImage(file = 'poster.png')
# Tạo một widget Label và hiển thị ảnh
label = Label(display, image=image_tk)
label.pack()

#show_img = Button(display, text = '', font = ('Time New Roman',11), image = image_tk)
#show_img.place(x=0,y=0)
#Tạo vòng lặp của cửa sổ để thao tác
display.mainloop()
```

Hình 4.1.3 Tạo nút nhấn và các đối tượng

Tạo nút nhấn dẫn đến hàm "Start", cho hiển thị ảnh poster và label. Cuối cùng là hàm display.mainloop() giúp lặp lại cửa sổ hiển thị.

4.2 Nhận diện khuôn mặt và đếm số lần nháy mắt:

CVZone là 1 nền tảng cung cấp cho ta các tài nguyên để học tập, nghiên cứu về vision computer. Ở đây ta dùng các thư viện như mediapipe opencv như công cụ để làm final project

4.2.1 Đếm số lần nhắm mắt :

Bước đầu ta sẽ khai báo các thư viện :

```
import cv2                                        # Thư viện opencv
import mediapipe as mp                            # Thư viện nhận dạng khuôn mặt, theo dõi đối tượng
from cvzone.HandTrackingModule import HandDetector # Thư viện nhận dạng bàn tay và theo dõi các điểm dấu hiệu của tay trong hình ảnh hoặc video
from cvzone.FaceMeshModule import FaceMeshDetector # Thư viện nhận dạng khuôn mặt và xử lý tác vụ trên khuôn mặt
import winsound                                   # Thư viện âm thanh MP3 dùng cảnh báo, mở nhạc
import cvzone                                     # Thư viện cvzone
from tkinter import *                             # Tạo giao diện để bắt đầu chạy chương trình
from PIL import ImageTk,Image                     # Thư viện xử lý ảnh PILLOW hỗ trợ nhiều loại định dạng
from cvzone.PlotModule import LivePlot            # Thư viện tạo biểu đồ chạy trực tiếp trong quá trình thực thi
```

Hình 4.2.1 Các thư viện cần thiết

Trong đó:

- import cv2 là thư viện opencv công cụ để xử lý ảnh và video,...
- import mediapipe as mp là thư viện nhận dạng khuôn mặt, theo dõi đối tượng
- from cvzone.FaceMeshModule import FaceMeshDetector là module trong thư viện cvzone sử dụng để phát hiện và trích xuất các điểm mốc trên khuôn mặt từ ảnh hoặc video
- import cvzone cung cấp các tính năng và công cụ bổ sung trong lĩnh vực Thị giác máy tính và xử lý ảnh

Khởi tạo đối tượng để xử lý và phân tích hình ảnh từ video trong các bước tiếp theo

- Dữ liệu đầu vào : hình ảnh từ webcam thời gian thực & khai báo các hàm.
- Khởi tạo khai báo giá trị đầu tiên để thực hiện chương trình và bắt đầu quá trình đọc khung hình từ webcam thực hiện các tác vụ tiếp theo.

```
def Start():
    cap = cv2.VideoCapture(0)                  # Thay đổi giá trị 0 nếu muốn sử dụng video khác
    detectorEye = FaceMeshDetector(maxFaces=1) # Tạo đối tượng lưới trên khuôn mặt gán cho biến dectectEye khởi tạo tham số maxFaces là 1
    mpDraw = mp.solutions.drawing_utils        # Cung cấp các hàm và lớp để vẽ các hình dạng và đường kẻ lên các hình ảnh và video
```

Hình 4.2.2 Khai báo hàm Start

- Trong đoạn mã trên, đoạn code bắt đầu bằng việc khởi tạo một đối tượng cv2.VideoCapture(0) để mở video từ một nguồn dữ liệu. Tham số 0 đại diện cho webcam được kết nối với máy tính. Bạn có thể thay đổi giá trị 0 để sử dụng video từ nguồn dữ liệu khác.
- Tiếp theo, đoạn mã tạo một đối tượng FaceMeshDetector và gán cho biến detectorEye. Đối tượng này sử dụng mạng lưới khuôn mặt (face mesh) để phát hiện và theo dõi các điểm trên khuôn mặt. Tham số maxFaces=1 chỉ định rằng chỉ phát hiện và theo dõi một khuôn mặt duy nhất.
- Cuối cùng, đoạn mã import module drawing_utils từ mediapipe.solutions và gán cho biến mpDraw. Module này cung cấp các công cụ để vẽ các hình dạng và đường viền lên hình ảnh hoặc video.

Liệt kê các điểm mốc trên khuôn mặt và khởi tạo các giá trị đầu tiên để thực hiện chương trình :

```
idList = [22, 23, 24, 26, 110, 157, 158, 159, 160, 161, 130, 243]
ratioList = []
blinkCounter = 0
time_sleep_Counter = 0
counter = 0
color = (255, 0, 255)
dir = 0
while True:
    success, img = cap.read()                          # Đọc khung hình từ webcam
    imgRGB = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)      # Chuyển đổi khung hình đọc từ không gian màu BGR sang RGB
    img_eye, faces = detectorEye.findFaceMesh(img, draw=False) # Sau khi tìm thấy khuôn mặt chúng ta sẽ quay lại face =false
```

Hình 4.2.3 Quét các điểm có trong danh sách

Trong đó:

- idList: Một danh sách chứa các giá trị chỉ mục (index) của các điểm trên khuôn mặt. Các giá trị này có thể được sử dụng để xác định vị trí của các điểm quan trọng trên khuôn mặt, ví dụ như mắt, miệng, mũi, vv.
- ratioList: Một danh sách rỗng được sử dụng để lưu trữ tỷ lệ (ratio) hoặc thông tin khác liên quan đến các yếu tố của khuôn mặt. Trong ngữ cảnh cụ thể, không có giá trị nào được gán cho ratioList trong đoạn mã này, vì vậy nó vẫn là một danh sách rỗng.

- blinkCounter: Một biến đếm được khởi tạo với giá trị ban đầu là 0. Biến này có thể được sử dụng để đếm số lần nháy mắt hoặc sự kiện liên quan đến mắt trong quá trình xử lý khuôn mặt.
- time_sleep_Counter: Một biến đếm được khởi tạo với giá trị ban đầu là 0. Biến này có thể được sử dụng để đếm thời gian hoặc sự kiện liên quan đến giấc ngủ hoặc nghỉ ngơi trong quá trình xử lý.
- counter: Một biến đếm khác được khởi tạo với giá trị ban đầu là 0. Đây là một biến đếm tổng quát có thể được sử dụng để đếm các sự kiện hoặc thao tác khác trong quá trình xử lý.
- color: Một biến màu sắc được khởi tạo với giá trị RGB (255, 0, 255). Biến này có thể đại diện cho một màu sắc cụ thể được sử dụng để vẽ các đối tượng hoặc hiển thị thông tin trên hình ảnh hoặc video.
- dir: Nó dùng để lưu trữ thông tin về hướng chuyển động hoặc trạng thái. Một biến được khởi tạo với giá trị ban đầu là 0. Mục đích chính của biến này không được xác định trong đoạn mã hiện tại, vì vậy không thể nói chính xác nhiệm vụ của nó trong ngữ cảnh cụ thể này.
- Tiếp theo, đoạn code bắt đầu bằng việc sử dụng đối tượng cap (một đối tượng cv2.VideoCapture) để đọc khung hình từ webcam. Phương thức cap.read() được gọi để đọc một khung hình mới và trả về hai giá trị:
- success: Một giá trị boolean (True hoặc False) cho biết liệu việc đọc khung hình đã thành công hay không.
- img: Một khung hình (ảnh) đọc được từ webcam.
- Sau đó, đoạn mã chuyển đổi không gian màu của khung hình từ BGR sang RGB bằng cách sử dụng cv2.cvtColor và gán kết quả vào biến imgRGB. Một số thuật toán xử lý ảnh và Thị giác máy tính hoạt động tốt với không gian màu RGB, do đó việc chuyển đổi này thường được thực hiện để đảm bảo sự tương thích.
-

Tiếp theo, đánh dấu và hiển thị các điểm quan trọng trên khuôn mặt và vẽ các đường thẳng kết nối giữa các điểm đó trên khung hình.

- Chạy vòng lặp để quét các điểm trên khuôn mặt và định vị các điểm trên các vị trí trên khuôn mặt (mí mắt), sau đó vẽ đường nối các điểm chúng ta định vị và đưa ra các đường chúng ta đọc được .

```
if dir > 0 :        # Trong quá trình đếm ngón tay chúng ta sẽ đếm rồi xuất video nhận dạng khuôn mặt
    face = faces[0] # Gán giá trị face cho khuôn mặt trong list đầu tiên

    for id in idList:
        cv2.circle(img, face[id], 5,color, cv2.FILLED)

    leftUp = face[159]          # Điểm cao nhất của mí trên
    leftDown = face[23]         # Điểm thấp nhất của mí dưới
    leftLeft = face[130]        # Điểm ở ngoài cùng bên trái
    leftRight = face[243]       # Điểm ở ngoài cùng bên phải
    lenghtVer, _ = detectorEye.findDistance(leftUp, leftDown)       # Khoảng cách 2 mí mắt
    lenghtHor, _ = detectorEye.findDistance(leftLeft, leftRight)    # Chiều dài 2 mí mắt

    cv2.line(img, leftUp, leftDown, (0, 200, 0), 3)                 # Vẽ đường nối giữa 2 điểm 159 và 23
    cv2.line(img, leftLeft, leftRight, (0, 200, 0), 3)              # Vẽ đường nối giữa 2 điểm 130 và 243
```

Hình 4.2.4 Gán các điểm đặc biệt vào các biến để tính toán

- Thực hiện tính toán tỷ lệ độ dài mắt và áp dụng các điều kiện để phát âm thanh và thay đổi màu sắc của đường vẽ mắt dựa trên giá trị tỷ lệ và thời gian nhắm mắt.

```
ratio = int((lenghtVer / lenghtHor) * 100)        # Tỉ lệ độ dài theo chiều dọc / độ dài theo chiều ngang
ratioList.append(ratio)         # Thêm giá trị ratio vào cuối danh sách ratiolist
if len(ratioList) > 3:
    ratioList.pop(0)            # Nếu số phần tử trong ratioList > 3 sẽ xóa bỏ phần tử đầu tiên (0) của list
ratioAvg = sum(ratioList) / len(ratioList)      # Tính gía trị trung bình của các phần tử trong danh sách ratiolist

if ratioAvg < 35 and counter == 0:      # Nếu độ dài bé hơn 35 và đếm là 0 thì sẽ thời gian nhắm mắt sẽ được đếm += 1
    time_sleep_Counter += 1
    blinkCounter += 1
    if blinkCounter > 30  :             # Nếu thời gian nhắm mắt đếm giới hạn lớn hơn 30 thì sẽ phát nhạc
        winsound.PlaySound('sleeply.wav',winsound.SND_FILENAME)
        blinkCounter = 0
    if time_sleep_Counter >= 3 :        # Thời gian đếm nhắm mắt khi ngủ khi lớn hơn hoặc bằng 3 thì sẽ phát nhạc
        # Phát file nhạc
        winsound.PlaySound('warning.wav',winsound.SND_ASYNC + winsound.SND_LOOP)
    elif time_sleep_Counter < 3 :       # Nếu thời gian đếm nhắm mắt bé hơn 3 thì sẽ dừng phát nhạc
        # Stop file nhạc
        winsound.PlaySound(None,0)
    color = (0,200,0)                   # Chuyển đường màu mắt khi giá trị độ dài nhắm mắt vượt quá độ dài cho phép
    counter = 1
```

Hình 4.2.5 Tính toán tỉ lệ mí mắt và so sánh các trường hợp

- Điều kiện độ dài mắt vượt qua một ngưỡng và đếm là 0, thì sẽ có các hành động như đặt lại biến và dừng phát âm thanh. Các thông số và biến cũng được cập nhật và hiển thị trên khung hình.

```
elif ratioAvg > 35 and counter == 0 :   # Còn nếu độ dài lớn hơn 35 và đếm là 0 thì sẽ không đếm tiếp và sẽ dừng phát nhạc
    time_sleep_Counter = 0
    if time_sleep_Counter < 3 :
        # Stop file nhạc
        winsound.PlaySound(None,0)
if counter != 0:
    counter += 1
    if counter > 10:
        counter = 0
        color = (255,0, 255)
if time > 900 :
    blinkCounter = 0                #Khi thời gian quá dài, reset biến blink
    time = 0                        #Reset thời gian đếm
cvzone.putTextRect(img, f'Sleep: {time_sleep_Counter}', (0, 100), scale = 2,thickness=2 ,colorR=color)
cvzone.putTextRect(img, f'Blink: {blinkCounter}', (0, 150), scale = 2 , thickness=2 ,colorR=color)
```

Hình 4.2.6 Điều kiện đồ dài mắt

4.2.2 Phát hiện trạng thái mở miệng và mức độ mở miệng (ngáp) :

- Ở đoạn code này, tương tự như mắt bước đầu tiên chúng ta sẽ khoanh vùng tại các điểm quan trọng trên miệng, tính toán tỷ lệ chiều dài và lưu trữ các giá trị tỷ lệ trong danh sách.

```
# Mouth
for id in idListMouth:
    cv2.circle(img, face[id], 5,color, cv2.FILLED)

MouthUp = face[0]
MouthDown = face[16]
MouthLeft = face[185]
MouthRight = face[325]
lenghtVerMouth, _ = detectorMouth.findDistance(MouthUp, MouthDown)
lenghtHorMouth, _ = detectorMouth.findDistance(MouthLeft, MouthRight)
#cv2.line(img, MouthUp, MouthDown, (100, 200, 100), 3)
#cv2.line(img, MouthLeft, MouthRight, (100, 200, 100), 3)

ratioMouth = int((lenghtVerMouth / lenghtHorMouth) * 100)
ratioList_Mouth.append(ratioMouth)
```

Hình 4.2.7 Khoanh vùng các điểm trên miệng

- Theo dõi và xử lý quá trình ngáp dựa trên giá trị trung bình của tỷ lệ (ratio) của khuôn miệng

```
if len(ratioList_Mouth) > 3: # Kiểm tra phần tử trong list ở điều kiện
    ratioList_Mouth.pop(0)
ratioAvg_Mouth = sum(ratioList_Mouth) / len(ratioList_Mouth) # Tính giá trị trung bình
if ratioAvg_Mouth > 94 :
    yawn = yawn + 1
    time_sleep_Counter = 0 # Tránh trường hợp đang ngáp, hệ thống reo gây ồn
print(ratioAvg_Mouth)
if yawn > 60 : # Tương đương với 3 lần ngáp
    winsound.PlaySound('sleepy_stop_car.wav',winsound.SND_FILENAME)
    yawn = 0                # Reset biến ngáp
cvzone.putTextRect(img, f'Yawn: {(int)(yawn/20)}', (0, 200), scale = 2, thickness=2 ,colorR=color)
                            # yawn/20 : Lấy trung bình thời gian ngáp 1 lần xấp xỉ 20 vòng lặp
```

Hình 4.2.8 Tính toán tỉ lệ (ratio)

❖ Các thư viện và phương pháp thực hiện chương trình :

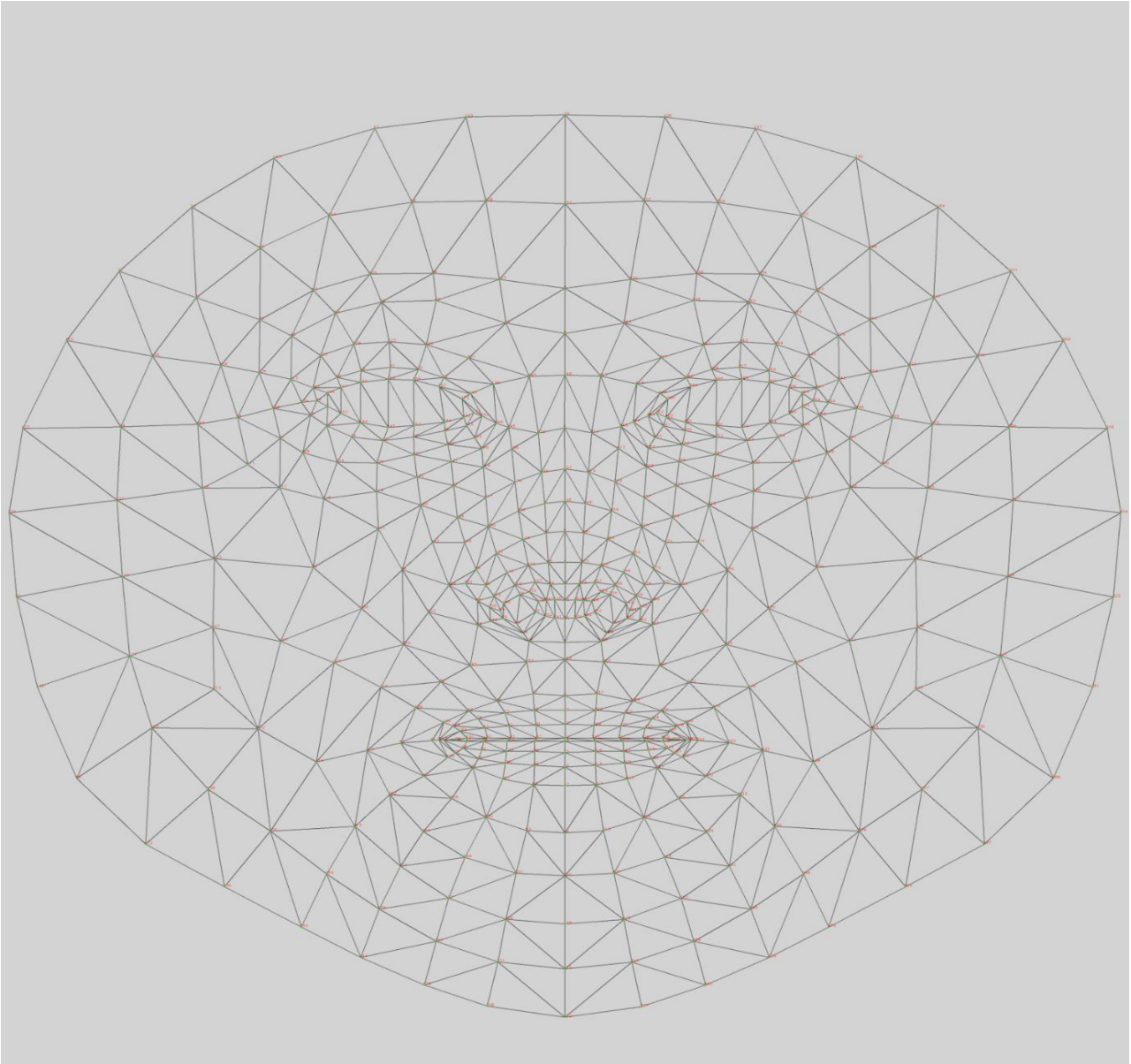
4.2.3 Các thư viện :

- CVZone: Là một thư viện Python được phát triển để hỗ trợ xử lý hình ảnh và thị giác máy tính. Nó cung cấp một loạt các công cụ và chức năng để xây dựng ứng dụng thị giác máy tính, nhận dạng khuôn mặt, xử lý video và hình ảnh, và tương tác với các thiết bị như webcam.
- CVZone có một số tính năng như:

- Điểm mắt: Cho phép nhận dạng và theo dõi các điểm mắt trên khuôn mặt.
- Điểm hình vuông: Cho phép nhận dạng và theo dõi hình vuông trên hình ảnh.
- Điểm đường tròn: Cho phép nhận dạng và theo dõi các đường tròn trên hình ảnh.
- Điểm khuôn mặt: Cho phép nhận dạng và theo dõi khuôn mặt trong hình ảnh hoặc video.
- Bộ đồ họa: Cung cấp các công cụ để vẽ các hình dạng và chú thích lên hình ảnh hoặc video.
- Đọc và ghi video: Cho phép đọc và ghi video từ webcam hoặc từ tệp video.

Để sử dụng thư viện "cvzone", bạn cần cài đặt nó trong môi trường Python của mình. Bạn có thể cài đặt nó bằng câu lệnh sau trong Command Prompt hoặc Terminal: “ pip install cvzone”.

4.3 Phương pháp dùng để thực hiện chương trình :



Hình 4.3.1 Hình ảnh các điểm mốc trên khuôn mặt dùng cvzone

Trong quá trình thực hiện dự án này, nhóm chúng em đã nghiên cứu và tham khảo từ các nguồn tài liệu trong và ngoài nước. Một số phương pháp sau tham khảo.

❖ **Phương pháp nhận dạng dựa trên các đặc trưng khuôn mặt**

Đây là phương pháp nhận dạng khuôn mặt dựa trên việc xác định các đặc trưng hình học của các chi tiết trên khuôn mặt như vị trí, diện tích, khoảng cách của mắt, mũi, miệng, ….và mỗi quan hệ giữa chúng ví dụ như khoảng cách giữa hai mắt.

Trong thư viện cvzone, “FaceMeshDetector” là một lớp trong mô hình “Face Mesh Module ” được sử dụng để nhận dạng và theo dõi các điểm trên khuôn mặt trong hình ảnh hoặc video. Mô hình này có thể phát hiện và trích xuất các điểm đặc trưng như điểm mắt, mũi, miệng, và các đường viền của khuôn mặt.

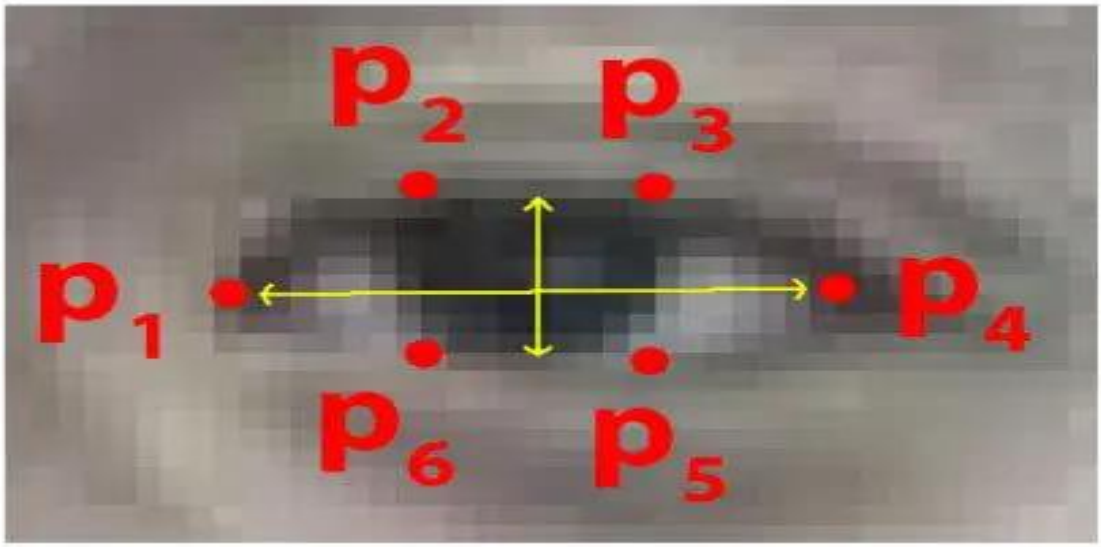
Vì vậy, dựa trên mô hình khai báo “FaceMeshDetector” ở trong python có thể giúp chúng ta phát hiện, trích xuất các điểm đặc trưng và tính toán khoảng cách hay độ dài của mắt hay của các bộ phận khác như miệng để đưa ra các ứng dụng chống ngủ gật hay các ứng dụng thực tế ngoài đời sống của chúng ta.

❖ Phương pháp xử lý hình ảnh truyền thống để tính toán chớp mắt thường liên quan đến một số kết hợp của:

1. Nội địa hóa mắt
2. Ngưỡng tìm lòng trắng con mắt
3. Xác định xem “vùng trắng” của mắt có biến mất trong một khoảng thời gian không (biểu thị nháy mắt)

Chúng ta có thể áp dụng tính năng phát hiện mốc trên khuôn mặt để khoanh vùng các vùng quan trọng trên khuôn mặt, bao gồm mắt, lông mày, mũi, tai và miệng.

Từ đó, chúng ta sẽ sẽ thảo luận về tỷ lệ khung hình của mắt và cách nó có thể được sử dụng để xác định xem một người có đang chớp mắt hay không trong một khung hình video nhất định.



Hình 4.3.2.2 Nhận diện tính toán các điểm ở mắt

Để tính toán tỷ lệ khung hình của mắt ta có công thức như sau:

$$\mathrm{EAR} = \frac{\|p_2 - p_6\| + \|p_3 - p_5\|}{2\|p_1 - p_4\|}$$

|Những điểm này được sử dụng để tính EAR trung bình giữa hai mắt do nháy mắt xảy ra khi cả hai mắt nhắm lại. Đối với mỗi khung hình, EAR cung cấp thông tin về mức độ mở của mắt. EAR thường cung cấp một số từ 0,20 đến 0,45. Để cải thiện biểu đồ trực quan hóa, EAR cho mỗi khung hình được nhân với 100. Kết quả là, các giá trị EAR được trích xuất từ mỗi người tham gia nằm trong khoảng từ 20 đến 4545

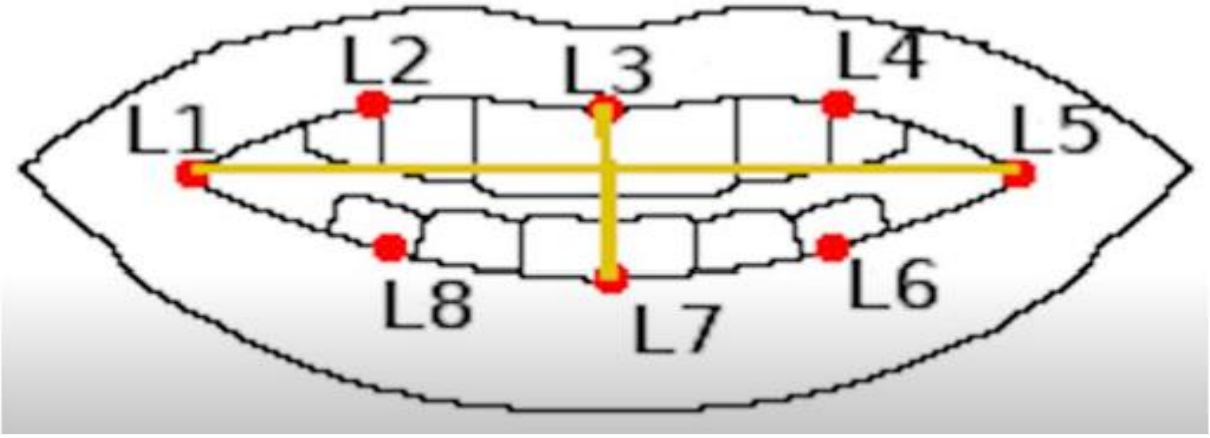
Vì vậy, nhìn vào hình chúng ta biết rằng khi mắt nhắm lại thì các điểm mốc ở trên mắt sẽ trùng nhau thành một đường thẳng nằm ngang và tỷ lệ khung hình của mắt giảm đáng kể, gần bằng không.

- **Phát hiện trạng thái nháy mắt:**

Nếu đôi mắt thay đổi từ trạng thái nhắm mắt sang mở mắt, thì hệ thống sẽ xác định đó là một cái nháy mắt. Và nếu trạng thái của mắt tiếp tục nhắm trong một khoảng thời gian nhất định (2 giây trong hệ thống này), thì mắt sẽ được phát hiện là nhắm.

$$LAR = \frac{|L2-L8|+|L3-L7|+|L4-L6|}{2|L1-L5|}$$

Fig 8: Calculating LAR for Yawn Detection



Hình 4.3.2.3 Tính tỉ lệ khuôn miệng

- Tương tự, phương pháp nhận dạng phần miệng và đánh giá mức độ mở miệng (ngáp):
- Chúng ta sẽ tính khoảng cách giữa các điểm mốc của các vùng xung quang miệng như ở công thức trên
- Ngáp là một quá trình mở miệng lớn liên tục, khác với nói chuyện với sự mở miệng lớn tạm thời. Mức độ mở miệng trên ngưỡng T được xác định trước cho N khung hình liên tục có nghĩa là xảy ra ngáp
- Ở các điều kiện cụ thể chúng ta đặt ngưỡng khi vượt qua ngưỡng hay ngáp quá mức thì hệ thống sẽ cảnh báo

4.4 Nhận diện đếm số ngón tay :

a. Khai báo các thư viện:

```
1  import cv2
2  import cvzone
3  import mediapipe as mp
4  from cvzone.HandTrackingModule import HandDetector
5  from cvzone.FaceMeshModule import FaceMeshDetector
6  import winsound          #phát MP3
7
```

Hình 4.4.1 Các thư viện để nhận diện bàn tay

- Dòng code 1: Khai báo thư viện CV2
- Dòng code 2: Khai báo thư viện cvzone
- Dòng code 3: Khai báo thư viện mediapipe để nhận dạng khuôn mặt và nhận dạng cử chỉ
- Dòng code 4: Đoạn code trên dùng để import lớp 'HandDetector' từ module 'HandTrackingModule' của thư viện 'cvzone'.

Việc sử dụng lớp 'HandDetector' cho phép ta nhận diện và định vị bàn tay trong hình ảnh, thông qua các phương thức có sẵn của lớp này. Lớp 'HandDetector' được cài đặt trong module 'HandTrackingModule' của thư viện 'cvzone', do đó ta cần import lớp 'HandDetector' bằng cách sử dụng câu lệnh 'from cvzone.HandTrackingModule import HandDetector' để sử dụng được các phương thức của lớp này trong bài toán của mình.

Sau khi đã import lớp 'HandDetector', ta có thể khởi tạo đối tượng thuộc lớp này để sử dụng việc nhận diện và định vị bàn tay trong ảnh (sử dụng phương thức và thuộc tính của đối tượng 'HandDetector').

b. Thiết lập ban đầu

```
27    handDetector = HandDetector(detectionCon=0.5, maxHands=2)
28    cap = cv2.VideoCapture(0)  # Thay đổi giá trị 0 nếu muốn sử dụng video khác
29    mpHands = mp.solutions.hands
30    mpDraw = mp.solutions.drawing_utils
31    hands = mpHands.Hands()
32
33    hand_count = []
34    idListEye = [22, 23, 24, 26, 110, 157, 158, 159, 160, 161, 130, 243]
35    idListMouth = [185,39,37,0,267,269,325,321,404,315,16,85,180,90]
36    ratioList_Eye = []
37    ratioList_Mouth = []
38    blinkCounter = 0
39    time_sleep_Counter = 0   # Đếm thời gian nhắm mắt
40    counter = 0              # Đếm số lần chớp mắt
41    color = (0, 0, 255)
42    dir = 0                  # Biến chia trường hợp ngón tay
43    yawn = 0                 # Biến đếm số lần ngáp
44    time = 0                 # Đếm thời gian thực hiện chương trình
45    maxframes = 12
46    num_frames1 = 0
47    num_frames2 = 0
48    num_frames3 = 0
49    detect_time = False
```

Hình 4.4.2 Thiết lập ban đầu

- Dòng code 27: Thiết lập ngưỡng độ chính xác để phát hiện bàn tay ‘detectionCon=0.5’. Giá trị 0.5 nghĩa là ta sẽ chỉ chấp nhận các tay nếu độ tin cậy của việc phát hiện của chúng lớn hơn 0.5, tức là 50%.

‘maxHands=2’ là số lượng tối đa của bàn tay được phát hiện trong một khung hình. Trường hợp này là 2.

- Dòng code 28: Mở Camera của Laptop
- Dòng code 29: Khai báo một biến ‘mpHands’, được gán giá trị là ‘mp.solutions.hands’. ‘mp.solutions.hands’ là một class trong module ‘mp’ sử dụng để phát hiện các điểm trên bàn tay (ví dụ như ngón tay, cổ tay,...) trong video. Tương tự như vậy, biến ‘mpHands’ này sẽ được sử dụng để đại diện cho lớp này khi gọi các phương thức, các thuộc tính và các biến của nó.
- Dòng code 30: Được sử dụng để vẽ các điểm và các đường đã được xác định
- Dòng code 31: Tạo một đối tượng "hands" kiểu HandDetector, mà được sử dụng để nhận diện các đặc điểm của bàn tay từ ảnh hoặc video đầu vào, bao gồm vị trí của các ngón tay, cổ tay, kiểu dáng của bàn tay, và các các hoạt động khác liên quan đến tay.
- Dòng code 33: Khởi tạo danh sách chứa số ngón tay trong mỗi khung hình.
- Dòng code 45: Khởi tạo giá trị số lần đọc khung hình là 12

- Dòng code 46-48: Khởi tạo giá trị số lần đọc ngón tay số 1, 2, 3 là 0
- Dòng code 49: Khởi tạo biến False

```
51    while True:
52        if voice_repeat == 0 :
53            winsound.PlaySound('start.wav',winsound.SND_FILENAME + winsound.SND_ASYNC)
54            voice_repeat = voice_repeat + 1
55        #setup
56        success, img = cap.read()
57        imgRGB = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
58        results = hands.process(imgRGB)
59
60        hand, img_hand = handDetector.findHands(img)
```

- Dòng code 51: Tạo vòng while để kiểm tra có bàn tay hiện diện trong khung hình.
- Dòng code 56: Chụp một khung hình từ nguồn video được chỉ định trong biến 'cap' và lưu trữ nó vào trong biến 'img'. Nếu thành công, 'success' sẽ là 'True'.
- Dòng code 57: Chuyển ảnh BGR sang RGB.
- Dòng code 58: dùng để xử lý hình ảnh ('imgRGB') và trả về kết quả detect tay trong hình ảnh đó ('results').
- Cụ thể, hàm 'hands.process()' sẽ nhận đầu vào là một bức ảnh RGB và sử dụng mô hình AI (bao gồm cả Deep Learning và Computer Vision) để phát hiện vị trí của các điểm đặc trưng trên tay trong ảnh đó. Kết quả được trả về gồm vị trí của các điểm đặc trưng trên tay, bao gồm cả các điểm của các đốt ngón tay, đầu ngón tay, cổ tay và dấu hiệu phát hiện của tay trong hình ảnh đó.
- Khi kết thúc lệnh này, biến 'results' sẽ chứa kết quả của việc xử lý hình ảnh và phát hiện tay, các kết quả được đưa vào để sử dụng cho bước tiếp theo của quá trình xử lý.
- Dòng code 60: Gọi một hàm để tìm kiếm tay trong ảnh `img` và trả về các thông tin về các điểm, vị trí và hướng của tay trong hình ảnh. Cụ thể, hàm này sử dụng mô hình mạng học đang dựa trên máy (machine learning) để phát hiện các điểm quan trọng trong tay, như các đốt ngón tay, mắt và cổ tay. Sau đó, các thông tin

này được trả về dưới dạng các đối tượng tham chiếu về một đối tượng Kiểu Hand (hand object type).

- Dòng code 'hand, img_hand = handDetector.findHands(img)' đồng thời khai báo giá trị của hàm 'findHands()' cho hai biến hàm số 'hand' và 'img_hand'. Biến 'hand' chứa toàn bộ các thông tin liên quan đến bàn tay được tìm thấy trong hình ảnh và nó được sử dụng để lấy vị trí và các thông tin liên quan đến các điểm trong tay, còn biến 'img_hand' chứa ảnh được xử lý với các đường viền và hướng cách tay được xác định trong hình ảnh gốc 'img'.

c) Vẽ khung tay :

```
61        #detect hand
62        if results.multi_hand_landmarks:
63            #Draw landmarks hand:
64            for handLms in results.multi_hand_landmarks:
65                for id, lm in enumerate(handLms.landmark):
66                    h, w, c = img.shape
67                    cx, cy = int(lm.x * w), int(lm.y * h)
68
69                    if id == 4:
70                        cv2.circle(img, (cx, cy), 3, (255, 0, 255), cv2.FILLED)
71            mpDraw.draw_landmarks(img, handLms, mpHands.HAND_CONNECTIONS)
72
```

Hình 4.4.3 Vẽ khung tay

- Dòng code 62: kiểm tra xem biến 'results' (kết quả từ bộ nhận diện tay của MediaPipe) có chứa thông tin về landmarks hay không. Nếu có, điều kiện sẽ trả về True, cho phép vẽ các landmarks trên ảnh.
- Dòng code 64: Trong vòng lặp đầu tiên, chúng ta duyệt qua từng tay trong danh sách các tay được phát hiện. Với mỗi tay đó, chúng ta duyệt qua từng landmarks của tay đó.
- Dòng code 65: Vòng lặp thứ hai giúp chúng ta nhận được thông tin về landmark của bàn tay.

- Dòng code 66: Chúng ta lấy kích thước của ảnh đầu vào bằng hàm ‘img.shape’, và lưu lại 3 thông số chiều sản phẩm của ảnh: chiều cao ‘h’, chiều rộng ‘w’ và số kênh ‘c’.
- Dòng code 67: Chúng ta tính toán tọa độ pixel tương ứng của các landmark bằng cách nhân tọa độ tương đối của chúng với kích thước tương ứng của ảnh. Cụ thể, để tính toán tọa độ theo trục x (‘cx’), ta nhân tọa độ tương đối ‘lm.x’ của landmark đó với chiều rộng ‘w’ của ảnh. Tương tự, để tính toán tọa độ theo trục y (‘cy’), ta nhân tọa độ tương đối ‘lm.y’ của landmark đó với chiều cao ‘h’ của ảnh.
- Dòng code 69-70: Sau khi tính toán được tọa độ của các landmarks trên ảnh, chúng ta tiếp tục kiểm tra xem landmark đó có chỉ số là 4 hay không (điểm trỏ). Nếu có, chúng ta sử dụng hàm ‘cv2.circle()’ để vẽ một hình tròn bán kính 3 pixel tại tọa độ đó trên ảnh.Trong hệ thống đánh số điểm định vị của Mediapipe Hand Landmarks, điểm 4 là điểm đầu tiên trên ngón tay trỏ khi tính từ phía cửa bàn tay
- Dòng code 71: Vẽ tất cả các landmarks và các đường nối giữa chúng trên ảnh.

d) Đếm số ngón tay :

```
73          #Count fingers :
74          hand_count = hand[0]     # Lấy tay đầu tiên (không quan tâm đến tay thứ 2)
75          fingers = handDetector.fingersUp(hand_count)   # Trả về số ngón tay trong tay
76          cvzone.putTextRect(img, f'Finger: {str(fingers.count(1))}', (0, 50), scale = 2,thickness=2 ,colorR=color)   # Vẽ số ngón tay
77
78          count = fingers.count(1)
79          if count == 1 :
80              num_frames1 += 1
81              num_frames2 = 0
82              num_frames3 = 0
83              if num_frames1 > maxframes :
84                  detect_time = True
85          elif count == 2 :
86              num_frames2 += 1
87              num_frames1 = 0
88              num_frames3 = 0
89              if num_frames2 > maxframes :
90                  detect_time = True
91          elif count == 3 :
92              num_frames3 += 1
93              num_frames2 = 0
94              num_frames1 = 0
95              if num_frames3 > maxframes :
96                  detect_time = True
97          else:
98              num_frames1 = 0
99              num_frames2 = 0
100             num_frames3 = 0
101             detect_time = False
```

Hình 4.4.4 Đếm số ngón tay

- Dòng code 74: Lấy tay đầu tiên trong danh sách các tay được phát hiện. Trong đoạn code này, ta không quan tâm đến tay thứ 2.
- Dòng code 75: Sử dụng hàm 'fingersUp()' của class 'HandDetector' để tính toán số ngón tay được uốn trong tay đó. Hàm 'fingersUp()' này sẽ trả về một danh sách chứa 5 phần tử tương ứng với 5 ngón tay. Nếu ngón tay đó được uốn, phần tử tương ứng trong danh sách sẽ là 0, ngược lại sẽ là 1.
- Cách thức đếm ngón tay trong hàm fingersUp() là một quá trình xử lý ảnh. Đầu tiên, hình ảnh được chuyển đến không gian màu HSV, sau đó chỉ giữ lại màu sắc tương ứng với màu da (được xác định bởi ngưỡng màu). Hình ảnh này được loại bỏ nền để chỉ còn lại ngón tay, sau đó phân tích kiến trúc ngón tay bằng cách xác định các điểm cực đại. Vận dụng kiến trúc của ngón tay, hàm 'fingersUp()' tính toán số ngón tay bằng cách đếm số điểm cực đại (số lượng khu vực cực đại của ngón tay) trên nền tảng máy tính. (xóa đoạn dưới)
- Dòng code 76: Ta sử dụng hàm 'putTextRect()' để vẽ số ngón tay tính được lên ảnh. Đây là một hàm để hiển thị văn bản lên ảnh với các đối số như: ảnh cần hiển thị lên, chuỗi văn bản, vị trí xuất hiện trên ảnh, tỷ lệ hiển thị, độ dày và màu của nét vẽ.
- Tham số đầu tiên 'img' là ảnh đầu vào cần vẽ hộp và văn bản lên đó.
- Tham số thứ hai 'f'Finger: {str(fingers.count(1))}'' là chuỗi văn bản cần hiển thị lên ảnh. Trong đó, 'fingers.count(1)' tính số lượng của các phần tử 1 trong mảng 'fingers'. Điều này có nghĩa là số lượng các ngón tay đã được phát hiện thông qua các điểm đầu vào trước đó.
- Tham số thứ ba '(0, 50)' xác định vị trí bắt đầu của văn bản. Điểm này nằm ở (0,50) trong hình ảnh.
- Tham số thứ tư 'scale' xác định kích thước của văn bản.

- Tham số thứ năm `thickness` xác định độ dày của đường viền hộp vẽ quanh văn bản.
- Tham số thứ sáu ‘colorR=color’ xác định màu sắc của văn bản và hộp được vẽ. ‘color’ ở chỗ này có thể là một giá trị màu bất kỳ, ví dụ như (255, 0, 0) để biểu thị màu đỏ.

- Dòng code 78: Gián số lượng ngón tay vào biến count
- Dòng code 79-101: Ta tạo điều kiện để tạo độ trễ khi nhận dạng các ngón tay.
- Ví dụ: Nếu ‘count == 1’ ta sẽ tăng giá trị ‘num_frames1’ lên 1 đồng thời thời gián lại 2 biến ‘num_frames2=0’ , ‘num_frames3=0’. Và đến khi vòng lặp này lặp đến khi ‘num_frames1 > maxframes’ thì biến ‘detect_time = True’. Tương tự với trường hợp ‘count == 2’ và ‘count == 3’. Nếu ‘count ’ khác các giá trị trên thì gián cho cả 3 ‘num_frames1=0’, ‘num_frames2=0’, ‘num_frames3=0’ và ‘detect_time = Flase’.

```
103            if detect_time and count == 1:
104                winsound.PlaySound('eyes-start.wav',winsound.SND_FILENAME )
105                dir  +=1
106            elif detect_time and count == 2:
107                winsound.PlaySound('goodbye.wav',winsound.SND_FILENAME )
108                dir = 0
109            elif detect_time and count == 3:
110                winsound.PlaySound('music_escape.wav',winsound.SND_FILENAME)
111                winsound.PlaySound('music.wav',winsound.SND_FILENAME + winsound.SND_ASYNC)
112                val = input("Enter any input to stop sound")
113                if ( val ) :
114                    winsound.PlaySound(None,0)
115
116                dir +=1
117            else :
118                dir *=2
119
```

Hình 4.4.5 Các câu điều kiện cho biến detect_time

CHƯƠNG 5 : KẾT QUẢ VÀ KẾT LUẬN

5.1 Vấn đề gặp phải và hướng giải quyết :

a) Không kết hợp được thư viện mediapipe cho nhận diện khuôn mặt và thư viện cvzone cho nhận diện bàn tay.

➡ Thay đổi nhận diện khuôn mặt sử dụng thư viện cvzone, khi đó việc lập trình, quản lý trở nên dễ dàng hơn.

b) Khi bàn tay ở các vị trí khác nhau làm cho việc nhận diện trở nên kém chính xác.

➡ Trong câu lệnh HandDetector(detectionCon=0.5, maxHands=2), ta thay đổi giá trị maxHands cho phù hợp.

c) Khi bàn tay vừa được nhận diện, phần mềm dễ bị đứng do độ nhạy cao ảnh hưởng đến việc báo động cho tài xế.

➡ Quét frame ảnh a lần (với a là số lần tuỳ chọn) để chờ tài xế giơ ngón tay

d) Không đồng bộ được hai hàm nhận diện bàn tay và nhận diện khuôn mặt.

➡ Tạo một biến để phân chia luồng (có thể xem lại ở lưu đồ thuật toán để hiểu rõ hơn), lợi dụng tính chẵn lẽ của số.

e) Khi âm thanh báo động phát, chương trình bị đứng.

➡ Trong câu lệnh winsound.PlaySound() ta thêm + winsound.SND_ASYNC để giúp âm thanh chạy đồng thời với chương trình mà không gây ảnh hưởng, trì trệ.

5.2 Kết quả đạt được :

Hình 5.2.1 : Giao diện bắt đầu của phần mềm

Hình 5.2.2 : Kích hoạt chế độ cảnh báo ngủ gật

5.3 Hạn chế của đề tài :

- Hệ thống chưa thể chứa quá 1 khuôn mặt.

- Hạn chế khi tắt âm thanh lúc phát.

- Hệ thống bị gián đoạn khi không nhận diện được toàn bộ điểm trên gương mặt.

- Camera chưa được rõ nét.

5.4 Hướng phát triển :

- Tích hợp thực tế vào hệ thống trên ô tô.
- Kết hợp với điều khiển động cơ, tác động đến hệ thống điều khiển trên xe.
- Xây dựng phần mềm thu thập dữ liệu về database.